

The Full Ledger

Everything in **The Disclosure Verifier** in one place — what was required, what was built to satisfy it, every phase in order, every hardening pass that came after the phases were "done," and the concepts and likely questions worth having cold before you explain any of it out loud.

↳ repository ↳ jump to requirements ↳ jump to Q&A prep

8 phases, 0–7	4 required pillars	260 automated tests	0/33 reconciler false positives*	96.7% real restatements caught
-------------------------------------	--	---	--	--

"We do not expect perfection. We expect honesty. A system that works 70% of the time and says so clearly is more valuable than one that works 95% of the time and hides the failure cases. Failure analysis is data. It reveals where your assumptions broke."

01

The requirements

The bootcamp capstone rubric names four required "pillars." Each one below states the requirement verbatim from the program's own capstone brief, where it was actually built, and the specific evidence that it was met — not just claimed.

Note on scope

This is a Bring-Your-Own-Project capstone

The brief names six example projects (Teaching Agent, Village Healer, Travel Companion, Disaster Nerve Center, Farmer's Almanac, Courtroom Companion) — The Disclosure Verifier is not one of them. Self-proposed capstones are governed by the brief's own "Bring-Your-Own-Project Guidance" checklist, in addition to the four pillars below:

COMPLEXITY	"requires decomposition into 2+ agents or a multi-step reasoning pipeline" — Coordinator + Retrieval/Extraction/Verification agents. MET
TOOLS	"at least 3 MCP tools or external API integrations" — exactly 3: Filing Retriever, MD&A Extractor, Numerical Reconciler. MET
OPTIMIZATION	"at least one DSPy signature with measured improvement over baseline" — the extraction signature, three optimizers tried. MET
FINE-TUNING	"a clear task-specific reward signal and GRPO optimization loop on a small model!" — the Reconciler-derived reward, Qwen2.5-7B-Instruct. MET
EVALUATION	"a quantitative metric and a test set" — held-out sets at every stage, hand-labeled before any DSPy code existed. MET

Pillar 1

3 TOOLS

Real MCP tools

"Real MCP tools" — independently callable, not stubs, not mocked in the tests that matter.

WHAT	Three independent MCP servers, each a real, separately testable capability: a Filing Retriever (pulls 10-K/10-Q filings + XBRL company-facts from live SEC EDGAR), an MD&A Extractor (parses filing HTML into cited prose paragraphs), and a Numerical Reconciler (checks one structured claim against XBRL facts across 4 comparison types).
WHERE	<code>tools/filing_retriever.py</code> , <code>tools/mdna_extractor.py</code> , <code>tools/numerical_reconciler.py</code> — built in Phases 1–3.
EVIDENCE	Each tool has its own live-network test tier (<code>pytest -m network</code>) hitting real EDGAR, not fixtures. The Reconciler's own tool <i>contract</i> was independently audited by reading its docstring "as the calling model would" — which is how a real, second bitemporal bug (a missing <code>as_of</code> parameter on the standalone tool call) was caught.

Pillar 2

HONEST NULL RESULT

DSPy-optimized prompt, measured

"A DSPy-optimized prompt with a measured baseline delta" — optimized, and proven against a hand-written baseline, not just used.

WHAT	The claim-extraction signature (<code>ExtractClaims</code>) is a <code>dspy.ChainOfThought</code> module, compiled with <code>BootstrapFewShot</code> and, separately, <code>GEPA</code> , scored against a hand-written baseline prompt on the same task.
WHERE	<code>eval/dspy_extractor.py</code> (signature), <code>eval/compile_and_save.py</code> (optimizer), <code>eval/run_comparison.py</code> / <code>eval/run_gepa.py</code> (measurement) — built in Phase 4.
EVIDENCE	F1 0.730 (baseline) → 0.784 (DSPy zero-shot) → 0.784 (BootstrapFewShot, identical) → 0.800 (GEPA, +0.016). The honest finding: reasoning (<code>ChainOfThought</code>) drove essentially the entire gain; optimization on top added a delta smaller than its own noise floor at every attempt. A fresh, never-touched holdout (AMZN/AAPL) later made the baseline-vs-DSPy comparison itself noise-floor-clearing for the first time: +0.355 F1 at n=82.

Pillar 3

HIERARCHICAL, NOT AUTONOMOUS

Justified multi-agent architecture

*"You must design an orchestration pattern that routes reasoning to tools, aggregates results, and decides when to stop... your choice must be justified and tested." The brief explicitly names **"Hierarchical multi-agent: a coordinator agent routes to specialist agents"** as one recognized pattern — this project's own architecture matches that description directly.*

WHAT	A Coordinator routes a strict, deterministic pipeline — Retrieval → Extraction → Verification — with a budget and checkpoint/resume harness, deliberately <i>not</i> a single autonomous agent loop or ReAct-style free reasoning. The argument: the pipeline's own sequencing never needs an LLM's judgment (you always retrieve before you extract, always extract before you verify); what actually needs deciding — skip an empty chunk, flag an unresolvable claim — is ordinary code, not a reasoning task. The one place the model's judgment is genuinely required (extracting claims from prose) is exactly the one DSPy-optimized step.
WHERE	agents/coordinator.py , agents/retrieval_agent.py , agents/extraction_agent.py , agents/verification_agent.py , agents/checkpoint.py — built and proven against mock tools with scenario traces (the brief's own required testing method) in Phase 5, load-tested for real in Phase 6.
EVIDENCE	The brief requires reporting "the number of tool calls, the average reasoning steps, and the time-to-first-action" — instrumented at runtime in every agents/schema.py run summary, not just for this section. Phase 6's final run: 128 tool calls, 119 reasoning steps across 5 companies, zero crashes. Budget enforcement and checkpoint/resume were also exercised against a real 215-chunk 10-K — a forced stop mid-document, a real on-disk checkpoint, a resume that skipped already-processed chunks and continued forward.

RLVR/GRPO fine-tuning, noise-floor-checked

The brief's own 6-step checklist: (1) choose a task-critical component, (2) design a verifiable reward, (3) collect 50–200 training examples, (4) fine-tune a small model with LoRA, (5) apply GRPO with G=8 samples per query, (6) report baseline vs. fine-tuned accuracy.

1 • TASK	Reconciliation- <i>reasoning</i> — given already-resolved claim values, reason to a verdict. Chosen deliberately: concept/period resolution is deterministic code, not a capability gap; this is the one place a small model's own reasoning is actually being tested.
2 • REWARD	The deterministic Reconciler itself, pre-audited (15/15 correct, 0 false-"consistent") before any GPU time was spent trusting it as a reward oracle. Reward-shaping recorded in docs/phase7-reward-design.md before training, not after.
3 • DATA	1,308 → 2,697 training examples across three dataset iterations (starting at 1,612 total examples, growing to 3,403) — well above the 50–200 the brief calls for, built entirely from real reconcile() calls, never hand-labeled.
4 • MODEL	Qwen2.5-7B-Instruct , QLoRA rank 32 via Unsloth — the exact first model the brief lists as an example choice.
5 • GRPO	Honest, documented deviation: NUM_GENERATIONS = 6 , not the brief's suggested G=8 — "tuned down from TRL's default (8) for a single 24GB card" (phase7/train_grpo.py), because training ran on a single RTX 4090, not the RTX 5090/32GB card the original planning doc assumed. A deliberate memory-vs-signal tradeoff, reasoned about in code comments and phase7/README.md , not an oversight.
6 • REPORT	Three full runs, each baseline vs. trained vs. delta, each checked against its own 95% noise floor. Final run: accuracy 0.820 → 0.865 , false-consistent rate 0.108 → 0.024 , both clearing noise floor at n=706. One category (bps_change) stayed a real, reported null despite two dedicated follow-ups — the honest limitation, not smoothed over.

02 Architecture

One sentence: retrieve real filing text and real structured data, extract discrete checkable claims from the prose with an optimized LLM step, resolve each claim's free text to an exact XBRL concept, then reconcile deterministically — no LLM judgment in the arithmetic, ever.

flowchart LR

subgraph Retrieval

A["Filing Retriever"] -->|"filings + XBRL facts"| D

B["MD&A Extractor"] -->|"cited prose chunks"| D

end

D["Coordinator"] --> E["Extraction Agent"]

E -->|"DSPy ChainOfThought, optimized"| F["ExtractedClaim"]

F --> G["Verification Agent"]

G -->|"resolver: text to XBRL concept"| H["Numerical Reconciler"]

H -->|"consistent / inconsistent / unverifiable"| I["Cited Report"]

D -. "budget + checkpoint" .-> D

LAYER	WHAT IT DOES	FILE
Filing Retriever	Pulls 10-K/10-Q filings + XBRL facts from EDGAR	<code>tools/filing_retriever.py</code>
MD&A Extractor	Extracts prose paragraphs from filing HTML, cited to accession number	<code>tools/mdna_extractor.py</code>
Numerical Reconciler	Checks a structured claim against XBRL facts, 4 comparison types	<code>tools/reconciler.py</code>
Extraction Agent	DSPy-optimized signature — prose → structured claims	<code>eval/dspy_extractor.py</code>
Verification Agent	Resolves free-text claims to exact XBRL concepts, then reconciles	<code>agents/verification_agent.py</code>
Coordinator	Routes the pipeline; enforces a budget; checkpoints for resume	<code>agents/coordinator.py</code>
RLVR/GRPO fine-tuning	Trains reconciliation-reasoning via the Reconciler-derived reward	<code>phase7/train_grpo.py</code>

The 4 comparison types the Reconciler handles: `absolute` (a stated dollar/unit figure), `absolute_change` (a stated dollar delta between periods), `growth_pct` (a stated relative percent change), and `bps_change` (a stated point-change in a ratio that's itself already a percentage — a margin, a rate). Each resolves through `agents/resolver.py` to an exact XBRL concept before the arithmetic ever runs; if it can't resolve confidently, the verdict is `unverifiable`, never a guess.

03 Phase by phase

Built in order, 0 through 7. Each phase is tagged with the pillar(s) it primarily serves.

00

Scaffolding

Repo layout, MCP boilerplate, CI skeleton — the empty pipes everything else runs through, wired up first so every later phase lands on a working test/CI harness instead of building one retroactively.

FILES `.github/workflows/ci.yml`, `pyproject.toml`

01

Filing Retriever PILLAR 1

Real MCP tool #1: pulls 10-K/10-Q filings and XBRL company-facts straight from live SEC EDGAR — no fixtures. This is the "ground truth" side of the whole project; every later reconciliation ultimately traces back to a fact this tool actually fetched.

FILES `tools/filing_retriever.py`, `tools/edgar_client.py`, `tools/xbrl_parser.py`

02

MD&A Extractor PILLAR 1

Real MCP tool #2: parses raw filing HTML down to cited prose paragraphs (the "Management's Discussion & Analysis" section, where the checkable dollar claims actually live). Replaced an earlier transcript-based retrieval approach entirely — the project pivoted to filing HTML directly rather than earnings-call transcripts.

FILES `tools/mdna_extractor.py`, `tools/mdna_parser.py`

03

Numerical Reconciler PILLAR 1

Real MCP tool #3, and the load-bearing piece of the whole project: given one structured claim and a set of XBRL facts, decide `consistent` / `inconsistent` / `unverifiable` — pure deterministic arithmetic, zero LLM judgment. Every downstream trust claim ("does this system actually catch wrong numbers") reduces to this function being right.

FILES `tools/reconciler.py` , `tools/numerical_reconciler.py`

04

Claim Extraction + DSPy PILLAR 2

The one genuinely LLM-judgment step in the pipeline: turn a paragraph of prose into structured `ExtractedClaim` objects (metric, value, unit, period, comparison type). Built as a `dspy.Signature` specifically so it could be optimized and measured against a hand-written baseline — see Pillar 2 above and the DSPy deep-dive in §04.

FILES `eval/dspy_extractor.py` , `eval/baseline_extractor.py` , `eval/labeled_claims.jsonl`

05

Orchestration skeleton + mock testing PILLAR 3

The Coordinator, budget enforcement, and checkpoint/resume harness — built and proven first against mocked agents so the harness logic itself (not network flakiness) is what's under test. A later pass deliberately re-verified four more harness properties by *reading the actual code* rather than re-trusting the original design doc.

FILES `agents/coordinator.py` , `agents/checkpoint.py` , `agents/schema.py`

06

Integration at scale PILLAR 3

First real, no-mocks run of the full Coordinator against 5 companies' actual 10-Ks — including GOOGL and AMZN, deliberately never used while building anything upstream, specifically to check generalization rather than confirm what was already tuned for. It found two real coverage gaps rather than a clean pass: MD&A heading detection didn't generalize past the 3 companies it was built against (fixed), and part of the resolver's "unresolved" claims were just missing dictionary entries for standard concepts, not a real data-source limit (18 more added and verified live). Checkpoint/resume was proven against a real 215-chunk filing, not just mock scenarios.

FILES `phase6/run_integration.py` , `phase6/exercise_checkpoint_resume.py` DOC `docs/phase6-results.md`

07

RLVR/GRPO fine-tuning PILLAR 4

Trains `Qwen2.5-7B-Instruct` (QLoRA rank 32, single RTX 4090) to reason through reconciliation given already-resolved values, using the audited Reconciler as the reward oracle. Three full runs: an initial 1,300-step run (accuracy 0.766→0.855), a retrain after fixing a data-generator bug that had starved the `bps_change` category (accuracy 0.820→0.858 on a larger, fresh 706-example holdout), and a third run testing whether more GRPO steps alone would fix `bps_change` — it didn't, but the rest of the model kept improving (0.865 accuracy, 0.024 false-consistent). Three real upstream packaging bugs (unrelated to the reward design) had to be worked around before training would even run.

FILES `phase7/build_dataset.py` , `phase7/reward.py` , `phase7/train_grpo.py` , `phase7/evaluate.py` DOC `docs/phase7-results.md` , `docs/phase7-reward-design.md`

04 Hardening & proof – what happened after "done"

The phases satisfy the rubric. What comes after is five separate investigations built to stress-test whether the system's own claims about itself actually hold — each one run against real data, each one reported with whatever it actually found, including the times that wasn't flattering.

Would this have caught a real restatement?

96.7%

SEC 8-K "Item 4.02" filings are a company telling investors its own past financials can't be relied on — real ground truth nobody had to construct. Scanned real XBRL data for amended-filing value disagreements, matched the results to real MD&A prose, then ran the actual `reconcile()` function twice per match: once `as_of` the original filing date, once `as_of` the restated date.

13,827 restatement fingerprints, 541 companies

122 matched to real prose, 16 companies

118/122 (96.7%) correctly flagged inconsistent once the restatement existed

Why a deterministic reconciler, not just an LLM?

TESTED, NOT ASSUMED

A naive baseline — same LLM, same claims, but raw unfiltered fact history and a plain-English "only use data before this date" instruction instead of code-enforced bitemporal lookup — run against the same two proven test sets.

11/15 naive vs. **15/15** reconciler on adversarial cases — naive missed a 1,000,000× unit error

50/102 (49%) naive vs. **102/102** reconciler on the real backtest's bitemporal pattern

naive falsely called **49/100** genuinely accurate claims "inconsistent" — bled future information into a past judgment

Does it cry wolf on clean filings?

0/33 → 0.0%

Ran the real Coordinator against 8 large tech companies confirmed to carry *no* restatement fingerprint (ADBE, CRM, ORCL, CSCO, INTU, IBM, QCOM, TXN). First pass: an apparent 25% false-positive rate. Investigated rather than reported at face value — it split into **eleven separate, confirmed root causes**, each diagnosed against real live data and fixed one at a time (a period-resolution bug that picked a 9-month YTD fact over the correct annual one; a verification bug that mis-paired "\$X compared with \$Y" sentences; a company that tags its own annual and quarterly filings with disagreeing fiscal years; XBRL instant-facts breaking a duration-based safeguard; an MD&A chunking bug that split one sentence across three chunks; a silent crash on one transient LLM response; a DSPy-compiled artifact freezing stale instructions; a token-limit truncation; and more).

25% → **0.0%** (0/33) reconciler false positives, given correct inputs

11 confirmed, individually verified root causes

Does 0% generalize, or was it overfit?

33.3%, ONE OPEN LIMITATION

The obvious follow-up question, asked of the project itself: 0/33 was measured only against the 8 companies the 11 fixes above were diagnosed against. Reran the identical check against a second, disjoint 8-company set never touched by any fix (NOW, PANW, CRWD, WDAY, SNPS, FTNT, INTC, MU). It found something real: a 45.5% false-positive rate, entirely in SNPS, from two causes — a segment/acquisition sub-figure mislabeled under the bare metric name (**fixed**), and a GAAP/non-GAAP ambiguity in "operating income" prose that an exact-match dictionary structurally can't disambiguate (**left open, documented**, not guessed at).

45.5% (5/11) → **33.3%** (3/9) after the fix — same, already-known cause every time

segment-misattribution bug: **fully eliminated**

What would full coverage actually cost?**\$11.31/YEAR**

The README's "why now" argument turned into a real number rather than left as a claim. Verification and retrieval are free (arithmetic and HTTP GETs); extraction is the only step with a real dollar cost — measured directly with real, cache-disabled calls against MSFT's actual 215-chunk 10-K MD&A.

1,537 avg prompt / **454** avg completion tokens per call, measured

\$11.31 to run extraction across every S&P 500 company's full annual 10-K, one year

05 Concepts to know cold

The load-bearing ideas — the ones worth being able to explain without looking anything up.

Bitemporal reconciliation (`as_of`)

A claim from an older 10-K must be checked against the data that existed *when it was made*, not a newer 10-Q's more recent figure and not a later restatement.

Threaded as an explicit `as_of` cutoff into both period selection and the reconciler's own fact lookup — two separate places it had to be fixed, found at two separate times.

Why it matters: it's the entire mechanism the real 96.7% restatement backtest exists to prove, and the exact thing the naive LLM baseline gets wrong 51% of the time.

Maker/checker precision ceiling

$$\Pr(\text{correct} \mid \text{accepted}) = \frac{pr}{pr + (1-p) \cdot f}$$
 — p = extraction precision, r = verifier recall, f = verifier false-positive rate. Formalizes why a near-zero-false-positive reconciler ($f \approx 0$) bounds trustworthiness almost independent of upstream extraction quality.

Why it matters: separates "is the arithmetic checker trustworthy" from "did extraction feed it the right inputs" — nearly every bug found this project long was upstream of the reconciler, never in its own arithmetic.

XBRL doesn't tag percent as percent

Rate concepts (effective tax rate) report `unit="pure"` — a decimal fraction (`0.19`), not `19` . A naive percent comparison either fails to match or compares 19.0 against 0.19 and reads as wildly wrong even when correct.

Instant facts vs. duration facts

Point-in-time balance concepts (e.g. remaining performance obligation) have `period_start=None` — any comparison logic that assumes a duration silently breaks for these, picking the wrong prior period.

Companies retag their own concepts

NVDA reported revenue under one XBRL tag through FY2022, then switched to another — the old tag never disappears, it just stops receiving new data. "First candidate that exists" silently locks onto stale data; the fix picks by data recency.

DSPy's compiled artifact freezes instructions

`eval/optimized_extractor.json` bakes the signature's `instructions` text at compile time. A live docstring edit has **zero effect** on the real extraction agent until `compile_and_save.py` is rerun — several early fixes this project silently never reached production, and happened not to matter for their own target cases purely by luck.

BootstrapFewShot's demo-freezing quirk

The optimizer walks the training set from the start and stops at the first 4 successful traces — new training examples appended later are never used as demonstrations. Every extraction fix ended up working through the signature's instruction text alone, never through new demos, once this was discovered.

Noise floor, everywhere

$SE(p) = \sqrt{p(1-p)/n}$, 95% half-width $1.96 \cdot SE$.
Applied to every reported delta in this project — several headline-looking improvements (BootstrapFewShot's F1 gain, GEPA's +0.016) turned out to sit *inside* their own noise floor and are reported as inconclusive, not rounded up to a win.

Why GRPO group size is 6, not 8

The capstone brief suggests $G=8$ samples per query. This project uses `NUM_GENERATIONS = 6` — a deliberate reduction, documented in the code itself, to fit GRPO's multi-completion rollouts in a single RTX 4090's 24GB VRAM (the original plan assumed a 32GB card).

Why it matters: it's a real hardware constraint reasoned about openly rather than silently worked around — exactly the kind of deviation worth naming before a reviewer finds it first.

GAAP vs. non-GAAP ambiguity

A company's plain-English "operating income" can mean the GAAP concept or a non-GAAP-adjusted figure, and an exact-match metric dictionary has no way to know which per company/year. Confirmed twice (CSCO, SNPS) — both times the real GAAP number moved in the *opposite direction* from the company's stated claim. Left open, not guessed at.

06 Reflection

The capstone brief names this explicitly as part of the final presentation: "a candid reflection on what you learned and what you would do differently." Most of that honesty is already scattered through §03 and §04 as it happened — this pulls it into one place.

What actually turned out to matter

Reasoning beat optimization, and it wasn't close. Switching to

`ChainOfThought` closed nearly the entire gap to a strong extractor;

`BootstrapFewShot` and GEPA on top of that added deltas smaller than their own noise floors, and on a fresh holdout, optimization actually made things worse. If the time budget had gone entirely to reasoning-prompt design and none to optimizer runs, the result would likely look the same.

Testing for overfitting is not optional busywork — it finds real bugs. The clean 0/33 result completely hid a real, fixable segment-misattribution bug that only the fresh-holdout test surfaced. A result that's only ever measured against the companies it was tuned on isn't evidence of anything beyond those companies.

A near-zero-false-positive checker is worth more than a stronger extractor. The maker/checker math isn't just theory here — nearly every real bug found across the whole project was upstream of the Reconciler, never in its own arithmetic. Investing in the checker's precision paid off more than investing further in extraction would have.

Naming a constraint openly beats working around it silently. The GRPO group size (6, not the brief's suggested 8) and the GPU (a 24GB RTX 4090, not the 5090 the brief assumes) are both real deviations from the plan — documented in code comments and in this guide rather than smoothed over. Compiled artifacts silently freezing stale prompts (the DSPy gotcha) was the opposite lesson learned the hard way: an undocumented, invisible constraint that cost real time before it was even noticed.

What would be done differently, with more time

Segment/dimensional XBRL coverage is the largest remaining gap — most claims resolve "unverifiable" because the company-facts API this project reads has no Axis/Member dimensional breakdown at all. A proper XBRL dimensional parser (reading the full instance document, not just company-facts) would likely unlock the majority of currently-unresolvable segment claims (Azure, Data Center, XBOX revenue) rather than correctly-but-unsatisfyingly declining them.

GAAP/non-GAAP disambiguation needs more than a dictionary. Two real, confirmed cases (CSCO, SNPS) show an exact-match metric-to-concept mapping structurally can't tell which one a company means. The now-labeled examples from both are a real starting training set for a small classifier or a second-pass LLM check purpose-built for this one ambiguity, rather than another dictionary entry that would just be wrong for the next company.

A frontier-model comparison point. Every result in this project — the DSPy deltas, the naive-baseline ablation, the RLVR gains — was measured against exactly one self-hosted `gpt-oss-20b` endpoint. Rerunning the naive-baseline ablation specifically against a frontier model would separate "is structure necessary" (this project's actual claim) from "is structure necessary for a model this size" (a narrower, unproven claim it currently can't rule out).

An explicit test of G=8 on better hardware. The GRPO group size was never A/B tested against the brief's suggested value — it was fixed at 6 for the whole project because of the GPU available. Access to the shared institutional cluster the brief mentions (RTX 5090/A100 nodes) would isolate whether group size itself was ever actually the limiting factor for the one category (`bps_change`) that never improved, rather than leaving that as an open, unconfirmed hypothesis.

07 Anticipated questions

Likely lines of questioning, answered the way the evidence actually supports.

Q Why not just hand an LLM the claim and the numbers and let it decide?

Tested directly, not assumed — see §04. The naive baseline gets adversarial cases right 73.3% of the time against the reconciler's 100%, misses a 1,000,000× unit error entirely, and gets the bitemporal reasoning wrong 51% of the time on the real backtest — **falsely flagging 49% of genuinely accurate claims** as inconsistent because it bleeds a later restatement's information backward into a judgment about the past. Telling a model "only use data before this date" in a prompt doesn't reliably make it do that; writing the cutoff into the fact-lookup code does.

Q How do you know the 0% false-positive rate wasn't just overfitting to the 8 companies you tested?

Asked that question of the project directly rather than stopping at the clean number: reran the identical check against a second, disjoint 8-company set untouched by any fix. It *wasn't* 0% — it was 45.5%, from a real bug. Fixed the fixable part, left the structurally-unfixable part (GAAP/non-GAAP ambiguity) open and documented, re-measured at 33.3%. The willingness to go looking for the overfitting — and report exactly what it found — is a bigger part of the answer than the final number.

Q Why RL (Phase 7) instead of just more DSPy optimization?

They target different gaps. Pillar 2's extraction task turned out to be almost entirely solved by reasoning (`ChainOfThought`) alone — optimization on top added nothing measurable. Phase 7's task (reason through reconciliation arithmetic given clean, already-resolved inputs) is where a 7B open model has a real, measured capability gap zero-shot (14.8% false-consistent rate) that no amount of prompt engineering closes cleanly — and the Reconciler supplies a perfect, pre-audited (0 false-consistent) reward signal, which is exactly the setup RLVR exists for.

Q What's still not solved?

Three honest, open items: (1) **Coverage** — most real claims resolve "unverifiable" because segment/product-level XBRL data doesn't exist anywhere in the company-facts API this project reads, a real data-source limit, not a bug. (2) **GAAP/non-GAAP ambiguity** — a structural limitation of exact-match metric resolution, confirmed on two companies, deliberately left open rather than papered over with a guess. (3) **Calendar-named quarters** ("the September quarter") still fall back to a default pick — translating them safely needs a company's fiscal-year-end, not available at the current resolution step.

Q What's "0/33" actually measuring?

The Reconciler's own false-positive rate *given already-correct, already-resolved inputs* — not the whole pipeline's end-to-end accuracy. That distinction is the maker/checker discipline applied throughout: separating "is the arithmetic checker trustworthy" from "did extraction and resolution feed it the right inputs." Nearly every real bug found this project long was in the second category, upstream of the reconciler — the reconciler's own arithmetic has never once been the thing that was wrong.

Q The brief suggests G=8 for GRPO — why does this project use 6?

A real hardware constraint, named openly rather than hidden: training ran on a single RTX 4090 (24GB VRAM), not the RTX 5090/32GB card the original planning doc assumed. `NUM_GENERATIONS = 6` is documented in `phase7/train_grpo.py` itself as "tuned down from TRL's default (8) for a single 24GB card," with the tradeoff it controls spelled out in `phase7/README.md`: GRPO's group-relative advantage needs enough samples per prompt to have signal, so this was a deliberate call, not an unexamined default.

Q What was the single most surprising bug?

The DSPy compiled-artifact freeze: `eval/optimized_extractor.json` bakes the signature's instruction text at compile time, so editing the live Python docstring had **zero effect** on the real production extraction agent until the compiler was explicitly rerun. Several earlier "fixes" this project had silently never reached production at all — and happened to not matter for their own target cases purely by luck, not because the fix worked.

Companion to the [case study](#) and the [interactive demo](#).

[View the repository](#) →